**Name: M. Sivateja**

**Hall.No:2303A52394**

**Batch: 41**

**Task 1 – Python to C++ Conversion PROMPT:**

Convert the given Python Student class into equivalent C++ code.

Include constructor, proper data types, access specifiers, and a main() function.

Ensure the output matches the Python program.

**CODE**

```
# Python Code (for reference)

# class Student:
#     def __init__(self, name, age, marks):
#         self.name = name
#         self.age = age
#         self.marks = marks

#     def display(self):
#         print("Name:", self.name)
#         print("Age:", self.age)
#         print("Marks:", self.marks)

# s1 = Student("Naizya", 20, 85)
# s1.display()
%%bash

cat > student_app.cpp << EOF
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    string name;
    int age;
    int marks;

public:
    // Constructor
    Student(string n, int a, int m) {
        name = n;
        age = a;
        marks = m;
    }

    // Display function
    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "Marks: " << marks << endl;
```

```
        cout << "Marks: " << marks << endl;
    }
};

int main() {
    // Creating object
    Student s1("Naziya", 20, 85);

    // Calling function
    s1.display();

    return 0;
}
EOF

g++ -o student_app student_app.cpp
./student_app
```

**OUTPUT:**

```
Name: Naziya
Age: 20
Marks: 85
```

**JUSTIFICATION:**

The Python class was converted into C++ by using proper data types, constructor, and access specifiers. The logic remains the same, and both programs produce the same output, confirming correct translation.

**Task 2 – Java to Python Function Conversion PROMPT:**

Convert the given Java isPrime() method into equivalent Python function  Ensure Pythonic syntax and correct logic Test the function with multiple inputs

**CODE:**

```
# Java Code (for reference)
# Java Code (for reference)

# public class PrimeCheck {
#     public static boolean isPrime(int n) {
#         if (n <= 1)
#             return false;

#         for (int i = 2; i <= n / 2; i++) {
#             if (n % i == 0)
#                 return false;
#         }
#         return true;
#     }

#     public static void main(String[] args) {
#         System.out.println(isPrime(7));   // true
#         System.out.println(isPrime(10));  // false
#     }
# }
def is_prime(n):
    if n <= 1:
        return False

    for i in range(2, int(n**0.5) + 1):  # optimized logic
        if n % i == 0:
            return False
    return True
```

```
def is_prime(n):
    if n <= 1:
        return False

    for i in range(2, int(n**0.5) + 1):  # optimized logic
        if n % i == 0:
            return False
    return True

# Testing with multiple inputs
print(is_prime(7))    # True
print(is_prime(10))   # False
print(is_prime(2))    # True
print(is_prime(15))   # False
print(is_prime(1))    # False
```

**OUTPUT:**

```
True
False
True
False
False
```

**JUSTIFICATION:**

The Java isPrime() method was converted into a Python function while keeping the same logic.Java syntax like data types and braces was replaced with Python's simple indentation.

The function was tested with multiple inputs to ensure correctness.The outputs matched the expected results, confirming successful translation.

**Task 3 – Pseudocode to Python Implementation**

**PROMPT:**

Convert the given pseudocode of bubble sort into Python code. Ensure correct logic and proper Python syntax. Test the program with sample input list

**CODE:**

```
# Pseudocode for Bubble Sort

# START
# Input list of numbers
# FOR i from 0 to n-1
#     FOR j from 0 to n-i-2
#         IF arr[j] > arr[j+1]
#             SWAP arr[j] and arr[j+1]
# END FOR
# Output sorted list
# END
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # swapping
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr
```

```
# Sample input
arr = [64, 34, 25, 12, 22, 11, 90]

# Function call
sorted_list = bubble_sort(arr)

# Output
print("Sorted list:", sorted_list)
```

**OUTPUT:**

```
Sorted list: [11, 12, 22, 25, 34, 64, 90]
```

**JUSTIFICATION:**

The given pseudocode for bubble sort was successfully converted into Python code. Nested loops were used to compare and swap adjacent elements. Python's simple syntax was used for swapping values efficiently. The program was tested with sample input and produced correct sorted output.

**Task 4 – SQL to Pandas Query**

**PROMPT:**

Convert the given SQL query into equivalent Pandas code. Apply condition and select required columnsTest using a sample DataFrame

**CODE:**

```python
# SQL Query (for reference)

# SELECT name, salary
# FROM employees
# WHERE salary > 50000;
import pandas as pd

# Sample DataFrame
data = {
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'salary': [60000, 45000, 70000, 30000]
}

employees = pd.DataFrame(data)

# Pandas equivalent query
result = employees[employees['salary'] > 50000][['name', 'salary']]

print(result)
```

**OUTPUT:**

```
      name  salary
0    Alice   60000
2  Charlie   70000
```

**JUSTIFICATION:**

The SQL query was converted into Pandas using DataFrame operations. The WHERE condition was implemented using boolean filtering. Column selection was done using double brackets in Pandas. The output matched the expected result, confirming correct translation. **Task 5 – Real-Time Application: Algorithm Translation Across**

**PROMPT:**

Convert the given Python linear search algorithm into equivalent C code Ensure proper handling of arrays, loops, and function structure Test both programs and verify output

**PYTHON CODE:**

```python
# Python Code

def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

arr = [10, 20, 30, 40, 50]
key = 30

result = linear_search(arr, key)

if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

**OUTPUT:**

```
Element found at index: 2
```

**C CODE:**

```bash
%%bash

cat > linear_search.c << EOF
#include <stdio.h>

int linear_search(int arr[], int size, int key) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == key) {
            return i;
        }
    }
    return -1;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int key = 30;
    int size = sizeof(arr) / sizeof(arr[0]);

    int result = linear_search(arr, size, key);

    if (result != -1)
        printf("Element found at index: %d\n", result);
    else
        printf("Element not found\n");

    return 0;
}
EOF

gcc -o linear_search linear_search.c
./linear_search
```

**OUTPUT:**

```
Element found at index: 2
```

**JUSTIFICATION:**

The Python linear search algorithm was successfully converted into C.

Differences in syntax and data types were handled appropriately.

Both programs were tested and produced the same output.

This confirms that the translation is correct and functionally equivalent.